Patterns in encrypted web traffic HTTPS

LORENZO GIANDOMENICO
LUDOVICO RIGOTTI
FEDERICO CAPITANI
DOMENICO DE PIETRO

1 Section 1 – Data Exploration and Pre-processing

The dataset consists of Tstat flow-level records of encrypted HTTPS/TCP connections, labelled with the domain name that generated each flow. The training set contains **147,863 flows** across **123 columns** and the test set contains **114,580 flows**, both with the same 123 columns. Of these, 120 are numeric Tstat features (all prefixed with _, further split into client-side `_c_*` and server-side `_s_*` statistics, plus a few connection-wide fields such as `_durat`), while `c_ip`, `time` and `label` are metadata. Both splits contain **26 domain-name labels**; the training set involves **738** distinct client IPs and the test set **788**. An initial audit found **zero missing values and zero duplicate rows** in either split, so no observations had to be discarded for data-quality reasons.

1.1 Merging Features at the IP and Domain-name Level

Both aggregated views are built the same way: the 120 numeric Tstat features are averaged (mean) within each group, and one popularity/diversity counter is appended. Concretely:

- **IP level**: grouping by `c_ip` yields **738 rows** (one per client), each carrying the per-feature mean over that client's flows, the number of flows generated by that client (`flow_count`), and the number of distinct domains it contacted (`domain_count`).
- **Domain level**: grouping by `label` yields **26 rows** (one per domain), each carrying the per-feature mean over that domain's flows, the number of flows it generated (`flow_count`), and the number of distinct client IPs that contacted it (`unique_ips`).

The mean was chosen over the median for simplicity and because it keeps the aggregated feature space on the exact same scale/units as the flow-level features, which matters for the correlation and PCA/t-SNE analyses in Sections 1.4 and 1.5 that are run identically at all three levels.

1.2 Class Imbalance and Feature Distributions

Table 1 shows the number of training flows per domain: the distribution is heavily imbalanced. `_other` (the catch-all class), `scdn.co` and `taboola.com` alone account for **23.7%**, **17.1%** and **14.2%** of flows respectively (55.1% combined), while the ten smallest classes each contribute under 1% (down to `vimeocdn.com` at 0.69%).

Domain	Flows	IPs	_s_bytes_all	_s_rtt_avg (ms)	_durat (ms)
_other	35,083	615	439,105	325.9	145,384
scdn.co	25,322	103	946,607	143.7	265,087
taboola.com	21,018	431	172,618	458.9	104,987
krxd.net	11,144	325	11,187	347.6	98,631
outbrain.com	5,637	349	59,901	599.5	166,837
githubusercontent.com	4,271	147	87,860	744.6	150,028
giphy.com	3,886	23	532,018	58.6	53,324
adnxs.com	3,839	233	31,287	329.3	87,007
reddit.com	3,457	48	196,562	183.4	81,178
ftcdn.net	3,455	11	159,067	27.2	4,427
everesttech.net	3,408	348	7,316	572.4	186,370
disqus.com	3,173	125	11,185	357.6	153,790

Table 1. Domain-level aggregates for the twelve most popular domains (mean of the raw, unscaled features).

Figure 1 shows the empirical CDF, on a log-x axis, of server bytes, server RTT and connection duration at all three levels (flow, IP, domain). The flow-level ECDFs are the most extreme (e.g. roughly 60% of flows carry under 10^4 server bytes, while the tail reaches 10^9); averaging within IPs and domains visibly smooths the curves because per-client/per-domain means wash out single-flow noise, but the underlying long tail persists at every level: the domain-level ECDF of `_s_bytes_all` still spans almost three orders of magnitude across only 26 points.

Authors· Contact Information: Lorenzo Giandomenico; Ludovico Rigotti; Federico Capitani; Domenico de Pietro

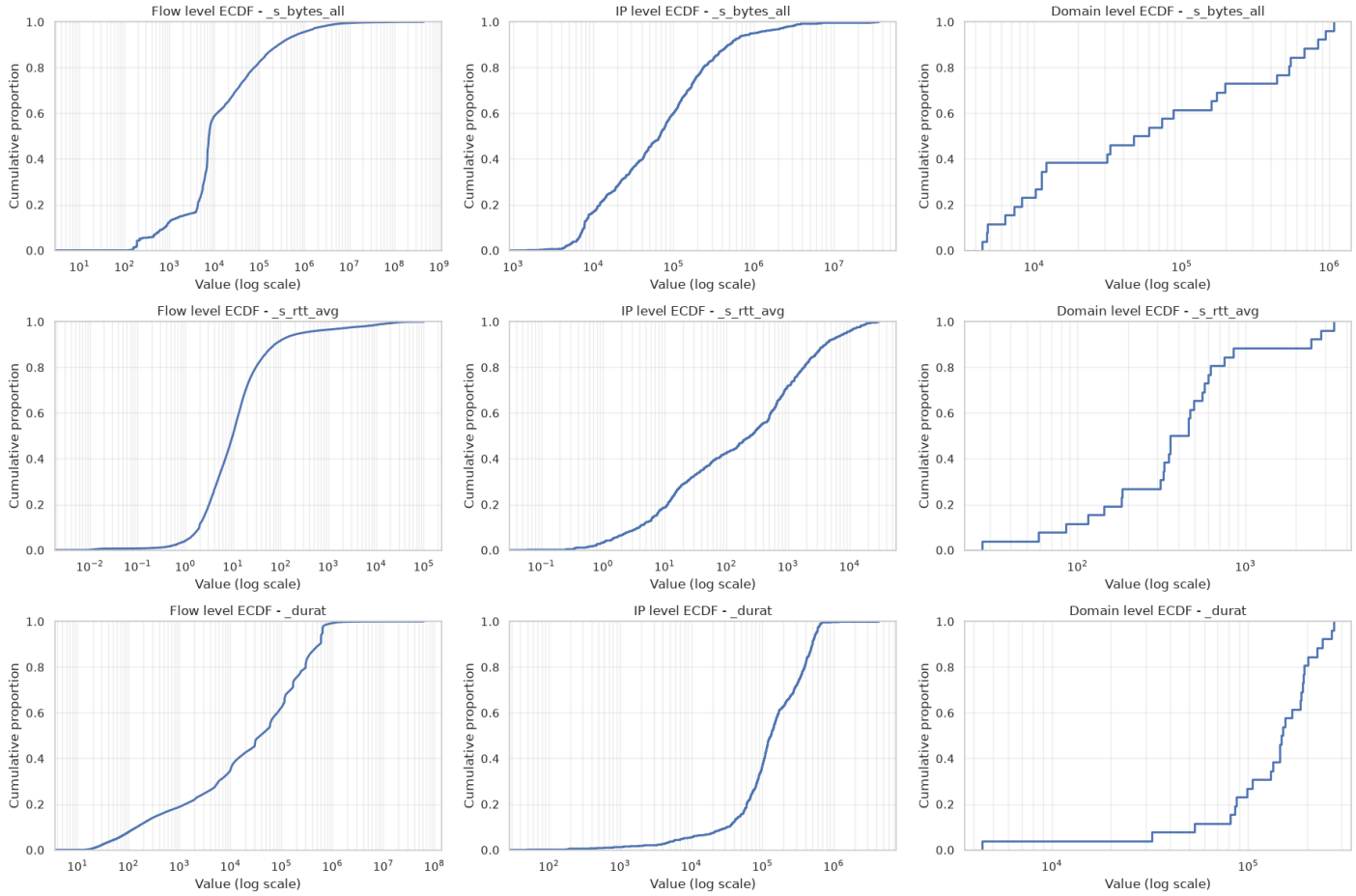


Fig. 1. ECDFs (log-x axis) of server bytes, server RTT and flow duration at the flow level (left column), IP level (middle column) and domain level (right column).

1.3 Bytes Transmitted and Round-Trip Time

Table 2 reports flow-level descriptive statistics about bytes transmitted and round trip time (RTT). Server-to-client traffic (`_s_bytes_all`) is far larger and more dispersed than client-to-server traffic, consistent with HTTPS being download-heavy: the median server flow carries 7,617 bytes versus 1,142 bytes for the client, but the 99th-percentile server flow already reaches 4.8 million bytes. RTT shows the opposite asymmetry: the median server-observed RTT (9.9 ms) is more than double the median client-observed RTT (4.0 ms), and its tail is far heavier (99th percentile 14.8 s vs. 25 ms), expected since server RTT reflects the full client–server path while client RTT often reflects only the local access network. Figure 2 visualises both: the top row shows marginal (log-scale) histograms; bottom-left is a scatter of bytes vs. RTT (log–log), confirming they are only weakly related; bottom-right is a per-domain boxplot of `_s_bytes_all` for the 12 most popular domains, already showing domain-specific payload profiles (e.g. `giphy.com` and `scdn.co` skew towards large media transfers, `krxd.net` towards small tracking-style flows), which is a first hint that flow statistics can discriminate between domains.

Feature	Mean	Median	p90	p95	p99	Max
`_s_bytes_all`	359,311	7,617	268,365	809,733	4.80×10^6	4.55×10^8
`_c_bytes_all`	5,112	1,142	5,487	10,199	35,595	2.54×10^7
`_s_rtt_avg`	426.4	9.9	75.5	272.7	14,801.2	100,682.3
`_c_rtt_avg`	7.1	4.0	12.6	14.2	25.1	661.7
`_durat`	152,768	38,608	565,617	645,136	886,633	6.24×10^7

Table 2. Flow-level descriptive statistics (raw units: bytes for the byte columns, ms for RTT/duration).



Fig. 2. Top: log-scale histograms of server bytes and server RTT (12k-flow sample). Bottom-left: server bytes vs. server RTT (log–log). Bottom-right: server bytes per flow for the 12 most frequent domains (outliers hidden).

1.4 Correlation Analysis

We computed the Pearson correlation between all numeric features at the flow, IP and domain level to look for redundant features. Across the three aggregation levels, the strongest correlations are dominated by the same types of redundant counters, with several pairs exhibiting near-perfect linear association ($|r| \geq 0.9998$). Two examples are: (i) Tstat sometimes tracks the same underlying events from different perspectives, e.g. `_c_pkts_data` (the client's data-packet count) and `_c_pktsize_count` (the count of recorded packet sizes); and (ii) TCP option features that are strongly coupled across the two endpoints, such as `_c_tm_opt` and `_s_tm_opt`. Figure 3 shows the resulting heat-maps: the pattern is essentially identical at all three levels. This strong, consistent redundancy is exactly why we have dropped one feature from every such pair or group during pre-processing (Section 1.6).

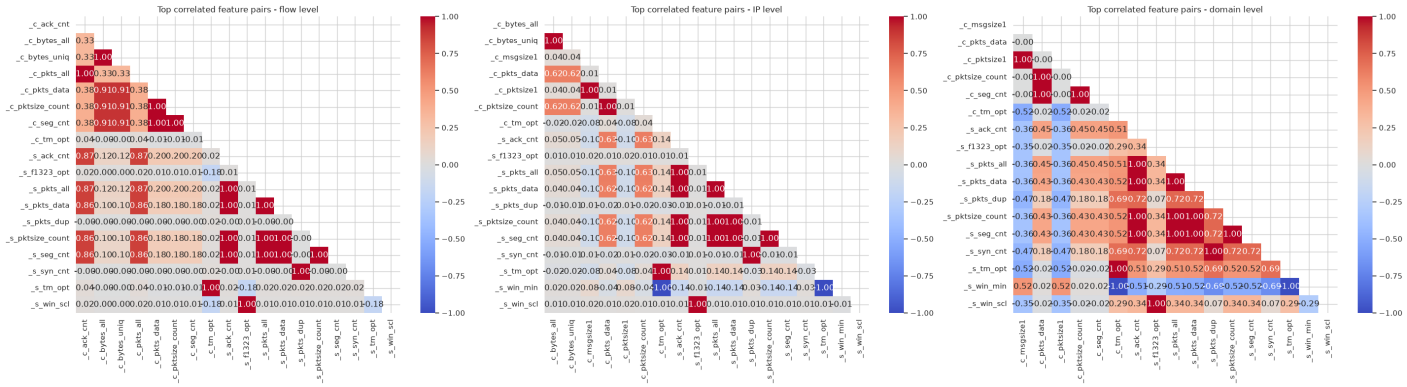


Fig. 3. Correlation heat-map restricted to the features in the twelve strongest pairs. (Left: flow level. Middle: IP level. Right: domain level)

1.5 PCA and t-SNE

To see whether domains are separable using only traffic statistics, we projected the features down to two dimensions with PCA and t-SNE, separately at the flow, IP and domain level. At flow level (computed on a 50,000-flow sample to keep t-SNE fast), Figure 4 (top row) shows that a single principal component already explains **90.7%** of the variance, and the first two together explain **97.2%** which is the same strong redundancy already found in Section 1.4. Both the 2D PCA and t-SNE plots show flows from every domain thoroughly mixed together with no visible separation: at the flow level, raw traffic statistics alone are not enough to tell domains

apart. At the IP level (middle row), the first two components explain a more modest **59.2%** of the variance (41.4% + 17.8%). Here, each point represents a client IP and the plot is therefore an unsupervised visualization of client-level traffic behavior rather than a domain-separability plot, since a client can contact multiple domains. At the domain level (bottom row), where each of the 26 points summarizes thousands of flows, the first two components explain **94.1%** of the variance (54.7% + 39.4%), and PCA shows a couple of domains standing clearly apart from the rest. Aggregating flows into per-domain averages therefore reveals structure that is less visible at the flow level.

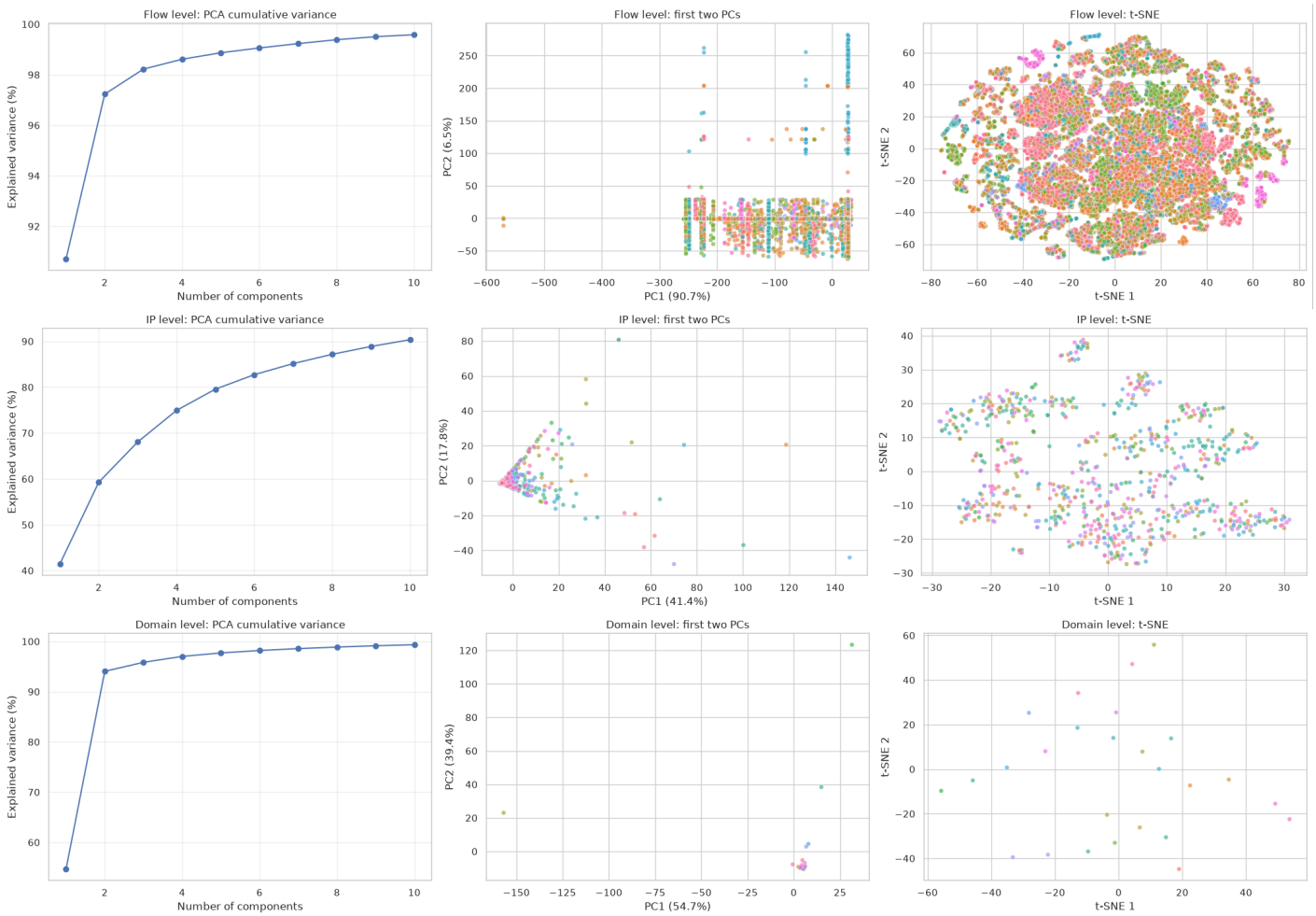


Fig. 4. PCA cumulative variance, first two PCs, t-SNE. (Top: flow level. Middle: IP level. Bottom: domain level)

1.6 Pre-processing

To prepare the dataset for machine learning and prevent data leakage, we apply four five pre-processing steps fitted exclusively on the training data:

(1) **Dropping Identifiers and Constant Features:** We remove the client IP address (`c_ip`) and flow timestamp (`time`). Keeping client IPs would cause models to memorize individual users instead of learning general traffic behavior, while timestamps would cause temporal overfitting. Columns with zero variance (constant values) are also removed because they contain no useful information.

(2) **Domain-Aware Feature Removal:** We remove features that can mislead the algorithms based on network domain knowledge: *Port Numbers (*`_c_port`, `_s_port`*)*. Client ports are chosen randomly by the client operating system in the ephemeral range (49152–65535), and server ports for HTTPS are almost always 443. Treating them as numerical features would only add random noise.

(3) **Log Transformation for Heavy-Tailed Features (log1p):** Network statistics like bytes, packet numbers, and flow durations vary by several orders of magnitude. We apply the transformation $f(x) = \ln(1 + \max(0, x))$. This reduces the impact of extreme outliers and skews, scales the values smoothly, and keeps exact zeros unchanged ($\ln(1 + 0) = 0$).

(4) **Removing Highly Correlated Features ($|r| > 0.95$):** Highly correlated pairs provide largely redundant information and can cause repeated aspects of the same traffic behavior to be represented multiple times. For example, after the log1p transformation, `_s_bytes_all` and `_s_bytes_uniq` have $r = 0.9996$, while `_c_pktsize_count` and `_c_pkts_data` have $r = 1.00$. The pruning is performed on the training set only. For each correlated group, one representative column is retained and the rest are removed, resulting in **31** of the 110 candidate features being dropped and leaving **79** features.
(5) **Robust Scaling:** The retained features are scaled with `RobustScaler`, fitted on the training set and then applied unchanged to the test set. This scaling uses robust statistics and is less sensitive to the extreme values that remain after the logarithmic transformation.

2 Section 2 – Supervised Classification

The main objective of this section is to build and evaluate a supervised machine learning pipeline to classify encrypted HTTPS network flows and predict the domain name visited by the user. The dataset contains traffic labeled across $C = 26$ domain classes. Because the communication is encrypted with TLS/HTTPS, application-level information (such as full URLs, HTTP request headers, and cookies) is completely hidden. Therefore, the models must rely entirely on transport-layer statistics (TCP/IP dynamics), such as packet counts, transmitted bytes, segment timing, and round-trip times extracted by the Tstat tool.

Preventing Data Leakage with Pipelines. To ensure a fair evaluation, all stateful preprocessing steps (such as `RobustScaler`, `StandardScaler`, PCA, and LDA) are encapsulated inside `sklearn.pipeline.Pipeline` objects. During 5-fold cross-validation, the scalers and projection matrices are fitted strictly on the training fold and applied to the validation fold using `.transform()`. This guarantees that no validation or test information leaks into the training phase.

2.1 Classifier Selection

We choose three machine learning algorithms representing distinct classification approaches:

- **Decision Tree (DT):** A simple, white-box model that splits the feature space using orthogonal decision rules. It is very easy to interpret, but unconstrained trees tend to overfit the training data.
- **Random Forest (RF):** An ensemble method that builds a collection of decorrelated decision trees using bagging and random feature selection. Averaging the predictions of multiple diverse trees significantly reduces variance and prevents overfitting.
- **K-Nearest Neighbors (KNN):** A non-parametric, distance-based algorithm that classifies a new flow based on the majority vote of its k closest neighbors in the feature space. While flexible, it suffers from the curse of dimensionality when many features are used.

Linear models, such as Logistic Regression, were excluded because encrypted network traffic exhibits complex, non-linear, and heavily overlapping boundaries between different domain classes.

2.2 Baseline Evaluation with Default Hyper-parameters

we first evaluate the three classifiers using their **standard default hyper-parameters**
from scikit-learn without manual tuning or depth limits:

- **Decision Tree (Default):** `RobustScaler()` + `DecisionTreeClassifier(max_depth=None, criterion='gini')`.
- **Random Forest (Default):** `RobustScaler()` + `RandomForestClassifier(n_estimators=100, max_depth=None)`.
- **KNN (Default):** `StandardScaler()` + `PCA(n_components=0.85)` + `KNeighborsClassifier(n_neighbors=5, metric='minkowski'`

Because the dataset has strong class imbalance (the background class `_other` represents a large fraction of the total flows), accuracy alone can be misleading. We adopt the **Weighted F1-Score** as the primary evaluation metric:

$$\text{Weighted } F_1 = \sum_{c=1}^{C} \frac{N_c}{N} \cdot F_{1,c}, \quad \text{where } F_{1,c} = 2 \cdot \frac{\text{Precision}_c \cdot \text{Recall}_c}{\text{Precision}_c + \text{Recall}_c} \tag{1}$$

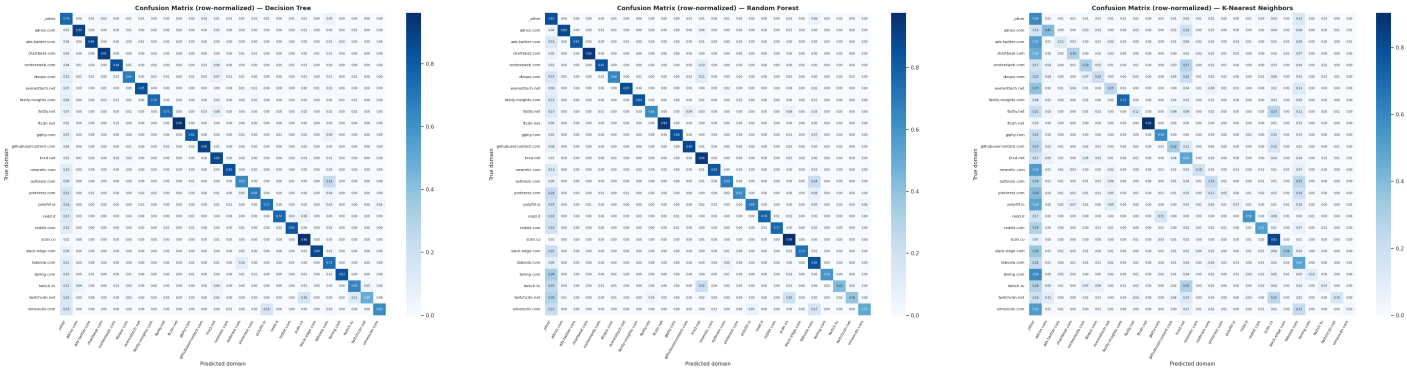
Table 3 summarizes the baseline results on the training set, 5-fold cross-validation, and the official unseen test set ($N = 114,580$). Figure 5 shows the corresponding normalized confusion matrices on the test set.

Diagnostic Analysis. Analyzing the baseline results reveals two distinct
behaviors:

- **Overfitting in Tree Models (DT and RF):** Both Decision Tree and Random Forest achieve a perfect training score ($F_1 = 1.0000$), but their performance drops substantially on the unseen test set (to 0.8039 and 0.8527, respectively). This creates a large generalization gap ($\Delta F_1 \approx 0.15 - 0.20$). Because leaf growth is unrestricted, the trees create isolated leaves for individual noisy samples, memorizing transient network artifacts (like temporary packet jitter or small bursts) rather than learning true protocol signatures.

Table 3. Baseline model performance under true default hyper-parameter settings.

Model	Train F_1	Test F_1	Train Acc	Test Acc	CV F_1 (± std)	Diagnosis
Random Forest	1.0000	0.8527	1.0000	0.8546	0.9279 ± 0.0014	Slight overfitting ($\Delta F_1 = 0.147$).
Decision Tree	1.0000	0.8039	1.0000	0.8033	0.9063 ± 0.0031	Severe overfitting ($\Delta F_1 = 0.196$).
KNN	0.7495	0.4840	0.7573	0.4977	0.6299 ± 0.0020	Severe underfitting / Curse of dimensionality.



(a) Decision Tree (Default) (b) Random Forest (Default) (c) KNN (Default)

Fig. 5. Row-normalized test set confusion matrices for baseline classifiers under default hyper-parameters.

- **Underfitting in Baseline KNN:** The default KNN model performs poorly on both training ($F_1 = 0.7495$) and test data ($F_1 = 0.4840$). This failure occurs because standard PCA is unsupervised: it projects data along directions of maximum total variance (which usually correspond to overall byte volume) rather than directions that separate the 26 domain classes. Furthermore, Euclidean distance in high dimensions becomes less effective because distances between points become very similar.

These diagnostic observations motivate the hyper-parameter tuning and dimensionality reduction improvements explored next.

2.3 Hyper-parameter Tuning via Validation Curves

To systematically regularize the models and improve generalization, we tune the hyper-parameters through **iterative validation curves** using 5-fold Stratified Cross-Validation on an internal 80% training split.

Using validation curves instead of an exhaustive grid search offers two clear benefits:

(1) **Clear Bias-Variance Diagnostics:** Validation curves directly show how training and validation scores evolve as a parameter changes, making it easy to identify the underfitting region, the optimal peak, and the overfitting region.
(2) **Computational Efficiency:** Instead of testing hundreds of combinations simultaneously, we tune the most influential structural parameter first, fix it in the pipeline, and then tune subsequent regularization parameters. This drastically reduces training time while avoiding combinatorial explosion.

Table 4 outlines the sequential search spaces and reasoning for each model.

Supervised LDA vs. Unsupervised PCA for KNN. A major architectural breakthrough for KNN is replacing unsupervised PCA with **Linear Discriminant Analysis (LDA)**. While PCA only looks at overall data spread, LDA uses the class labels to maximize the distance between different classes while minimizing the spread within each class. For our $C = 26$ classes, LDA compresses the features into exactly $C - 1 = 25$ dimensions designed specifically for class separation. This makes distance calculations between flows much more reliable.

Discussion of Tuning Results (Point 2: Performance Variation and Best Model). The validation curves (Figures 6, 7, 8) and final test scores (Table 6) highlight several important patterns:

- **Random Forest:** Figure 6 shows that increasing `n_estimators` steadily improves cross-validation performance up to 200–300 trees, where gains plateau. Setting `max_depth=30` maintains high expressive power while stabilizing variance. As a result, Tuned Random Forest achieves the highest performance overall, reaching a Test F_1 of **0.8537** (and Test Accuracy of **0.8556**), confirming it as the **Champion Model**.

Table 4. Sequential hyper-parameter search spaces and optimization rationale.

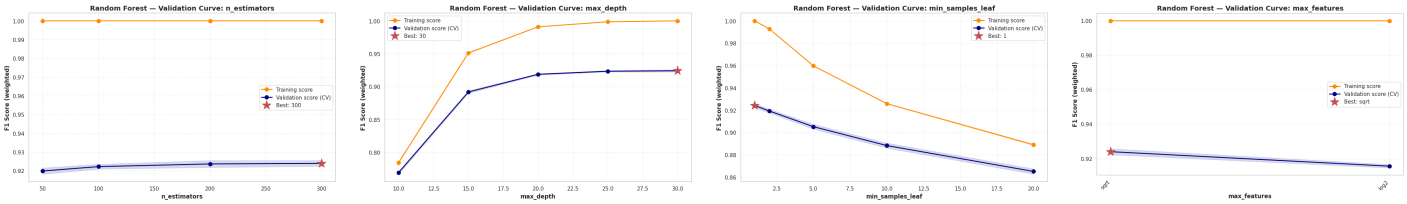
Model	Parameter	Search Space	Optimization Rationale
Random Forest	1° `n_estimators`	[50, 100, 200, 300]	Checks where ensemble variance reduction stabilizes.
	2° `max_depth`	[10, 15, . . . , 30]	Limits tree depth to control memorization of training noise.
	3° `min_samples_leaf`	[1, 2, 5, 10, 20]	Enforces statistical smoothing at leaf partitions.
	4° `max_features`	`['sqrt', 'log2']`	Controls tree decorrelation across random feature splits.
Decision Tree	1° `max_depth`	[5, 10, . . . , 30]	Finds the optimal tree depth balance between bias and variance
	2° `min_samples_leaf`	[1, 5, 10, 20, 50]	Eliminates fragile single-sample leaf splits.
	3° `criterion`	`['gini', 'entropy']`	Compares Gini impurity vs. Shannon entropy splitting.
KNN	1° `Projection`	`PCA(0.50-0.95), LDA`	Compares unsupervised PCA against supervised LDA separatio
	2° `n_neighbors`	[3, 5, 7, 9, 11]	Balances local boundary flexibility vs. regional smoothing.
	3° `weights`	`['uniform', 'distance']`	Gives closer neighbors higher voting weight ($w_i = 1/d_i$).
	4° `metric`	`['cosine', 'manhattan', 'minkowski']`	Tests Cosine and Manhattan distances against standard Euclide

Table 5. Optimal hyper-parameters identified via iterative validation curves (internal 80% split).

Classifier	Hyperparameter	Optimal Value	Best CV Weighted F_1
Random Forest	`n_estimators`, `max_depth`, `min_samples_leaf`, `max_features`	300, 30, 1, `'sqrt'`	**0.9241** (±0.0017)
Decision Tree	`max_depth`, `min_samples_leaf`, `criterion`	20, 1, `'entropy'`	**0.9109** (±0.0030)
KNN	`Projection`, `n_neighbors`, `weights`, `metric`	LDA, 3, `'distance'`, `'cosine'`	**0.7243** (±0.0016)

Table 6. Tuned model performance evaluated on the official test set ($N = 114,580$).

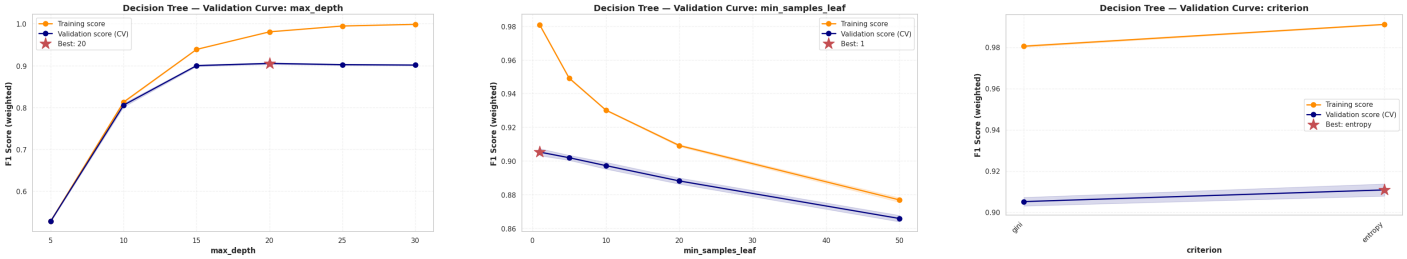
Model	Train F_1	Test F_1	Train Acc	Test Acc	Gap (ΔF_1)	Diagnosis
Random Forest (Tuned)	0.9998	**0.8537**	0.9998	**0.8556**	0.1461	Best overall test performer (Champion).
Decision Tree (Tuned)	0.9887	**0.8269**	0.9887	**0.8263**	0.1618	Substantial regularization gain (+2.30% F_1).
KNN (Tuned)	1.0000	**0.5698**	1.0000	**0.5726**	0.4302	Massive gain (+8.58% F_1 via LDA + Cosine).



(a) `n_estimators` (b) `max_depth` (c) `min_samples_leaf` (d) `max_features`

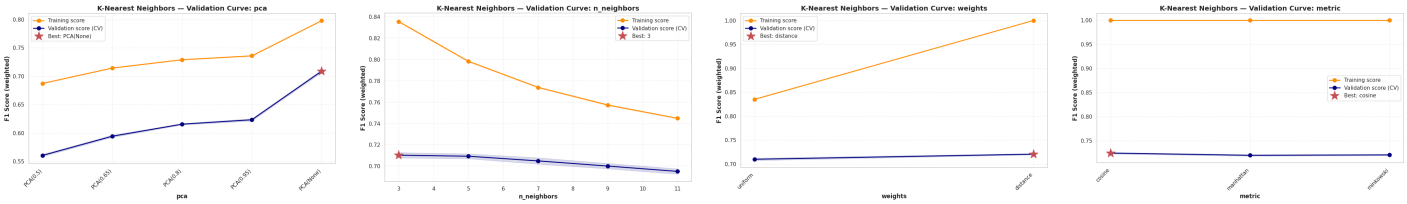
Fig. 6. Iterative validation curves for Random Forest across its four sequential hyper-parameters.

- **Decision Tree:** Bounding tree depth to `max_depth=20` and switching to Shannon entropy improves test performance by **+2.30%** (F_1 increases from 0.8039 to 0.8269). The validation curve for depth (Figure 7a) clearly shows that deeper trees continue to improve on the training set but plateau on the validation set, demonstrating the risk of memorizing noise without depth limits.
- **KNN:** The most dramatic improvement occurs in KNN. Replacing PCA with supervised LDA boosts the cross-validation score from ≈ 0.63 to 0.7243. Combined with distance weighting and the Cosine distance metric, the test F_1 increases by **+8.58%** (from 0.4840 to 0.5698). While KNN still lags behind the tree-based models, this result proves that supervised dimensionality reduction is essential for distance-based methods on network data.



(a) `max_depth` (b) `min_samples_leaf` (c) `criterion`

Fig. 7. Iterative validation curves for Decision Tree capacity constraints and splitting criteria.



(a) `PCA / LDA` (b) `n_neighbors` (c) `weights` (d) `metric`

Fig. 8. Iterative validation curves for KNN highlighting the impact of supervised LDA projection and distance metrics.

2.4 In-Depth Error Analysis

To understand the physical and network causes behind classification errors, we carry out a detailed error analysis on the **Champion Model** (**Random Forest Tuned**, Test $F_1 = 0.8537$) evaluated on the full test set ($N = 114,580$).

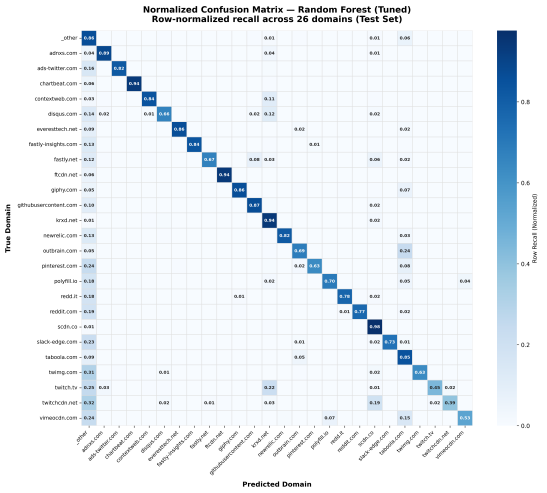
2.4.1 Multi-Class One-vs-Rest ROC and AUC Analysis. We compute One-vs-Rest (OvR) Receiver Operating Characteristic (ROC) curves for all 26 domains. Table 7 summarizes the AUC values for representative classes, and Figure 9 displays the normalized confusion matrix alongside the full set of 26 ROC curves.

Table 7. One-vs-Rest AUC summary for representative domain classes (Champion Model).

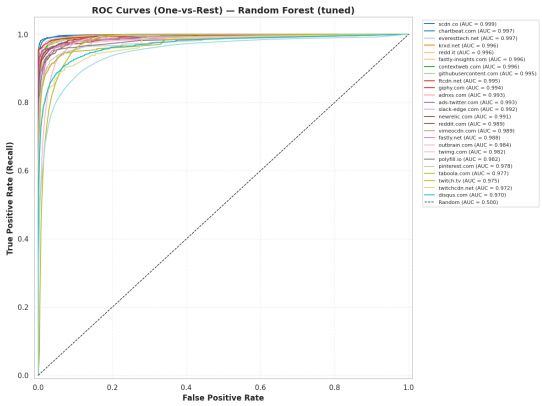
Domain Class	**Test Support (N_c)**	**OvR AUC**	**Physical / Network Signature Interpretation**
`scdn.co`	13,223	**0.9986**	Spotify audio streaming; steady packet pacing and high byte volume.
`chartbeat.com`	1,717	**0.9973**	Real-time analytics beacon; highly regular periodic heartbeat intervals.
`everesttech.net`	2,894	**0.9965**	Adobe ad telemetry; small payload request/response cycles.
`krxd.net`	11,171	**0.9961**	Salesforce tracking pixel; rapid transactional exchanges.
`githubusercontent.com`	4,477	**0.9952**	GitHub static assets; predictable single-GET transmission dynamics.
…	…	…	…
`outbrain.com`	4,426	**0.9839**	Recommendation widget; architectural overlap with Taboola.
`disqus.com`	3,246	**0.9704**	Embedded comment widget; variable scripts and background polling.
`_other`	34,777	**0.9507**	Heterogeneous composite background web traffic.
Macro Average	**114,580**	**0.9871**	Unweighted mean across all 26 classes.
Weighted Average	**114,580**	**0.9775**	Support-weighted mean across entire test partition.

Understanding the AUC vs. F1 Discrepancy. An important finding is that the champion model achieves an impressive Macro AUC of **0.9871** and Weighted AUC of **0.9775**, even though some minority classes have lower F1-scores. This demonstrates that the model is very effective at ranking domain probabilities. The lower F1-score in certain minority classes happens because the default decision threshold (0.5 via argmax) favors the majority class (`_other`) when class imbalance is extreme.

2.4.2 False Positive (FP) and False Negative (FN) Analysis (Point 3). To diagnose why certain flows are misclassified, we examine the classes with the highest False Positive and False Negative rates (Table 8) and analyze the most frequent confusion pairs (Table 9).



(a) Normalized Confusion Matrix



(b) ROC Curves (One-vs-Rest)

Fig. 9. Normalized confusion matrix and One-vs-Rest ROC curves across all 26 classes for the champion classifier.

Table 8. Domains generating the highest False Negatives and False Positives (Champion Model).

Domain Class	Test Support	TP	FP	FN	Precision Loss (FP Rate)	Recall Loss (FN Rate)
`_other`	34,777	29,904	6,409	4,873	17.65%	14.01%
`taboola.com`	15,764	13,351	4,020	2,413	23.14%	15.31%
`outbrain.com`	4,426	3,017	1,357	1,409	31.02%	31.83%
`disqus.com`	3,246	2,117	376	1,129	15.08%	34.78%
`krxd.net`	11,171	10,552	1,521	619	12.60%	5.54%
`pinterest.com`	1,542	970	71	572	6.82%	37.09%
`fastly.net`	1,455	931	80	524	7.91%	36.01%

Table 9. Most frequent pairwise classification confusions and physical network transport drivers.

Rank	True Domain (y_{true})	Predicted Domain (y_{pred})	Misclassified Flows	Physical Transport Driver
1	`_other`	`taboola.com`	2,237	Third-party ad widget traffic sharing common web patterns.
2	`taboola.com`	`_other`	1,389	Short or interrupted ad sessions that resemble generic web browsing.
3	`outbrain.com`	`taboola.com`	1,096	Direct competitors with nearly identical widget architecture and payloads.
4	`taboola.com`	`outbrain.com`	815	Bidirectional confusion between competing content recommendation widgets.
5	`_other`	`scdn.co`	488	Background media/podcast streams that mimic Spotify streaming pacing.
6	`disqus.com`	`_other`	485	Embedded comment widgets with variable scripts blending into web traffic.

Physical Network Causes of Misclassification. Our analysis reveals three primary network drivers of misclassification:

(1) **Competitor Mimicry (`taboola.com` vs. `outbrain.com`):** The largest source of specific cross-class confusion occurs between Taboola and Outbrain (1,096 Outbrain flows predicted as Taboola, and 815 Taboola flows predicted as Outbrain). Both services operate native content recommendation widgets embedded on news websites. They fetch small JSON feeds, display thumbnail grids, and use identical polling intervals. Because their transport-layer statistics (packet sizes, burst lengths, and flow durations) are nearly identical, the classifier struggles to separate them without access to unencrypted domain headers.

(2) **Overlap with Background Traffic (`_other`):** The background class contains a wide variety of unlabelled web traffic. Taboola and Disqus flows frequently get confused with `_other` (and vice-versa). When a user browses a web page and quickly navigates away, the resulting short, truncated TCP connection lacks the distinctive steady-state pacing of a complete session and looks like generic background web traffic.
(3) **Media Streaming Pacing (`scdn.co`):** Spotify media traffic achieves high recall because continuous audio playback generates distinct, steady-state TCP pacing with large packet payloads. The few false positives (488 flows from `_other` predicted as `scdn.co`) correspond to general audio, podcast, or video streams present in the background traffic that exhibit the same steady data throughput.

2.5 Final Model Leaderboard and Discussion

Table 10 compiles the final leaderboard ranking all six model configurations (the 3 baseline default models and their 3 tuned counterparts) evaluated on the official unseen test set ($N = 114,580$). Figure 10 visually compares the training and test Weighted F1-Scores to highlight the generalization gap across all models.



Fig. 10. Visual comparison of Weighted F1-Score on training and test sets across all six evaluated models.

Table 10. Definitive model leaderboard evaluated on the official test partition.

Rank	Model	Train F_1	Test F_1	Train Acc	Test Acc	Gap (ΔF_1)	Diagnosis
1	**Random Forest (Tuned)**	0.9998	**0.8537**	0.9998	**0.8556**	0.1461	Best generalisation (Winner).
2	**Random Forest (Default)**	1.0000	**0.8527**	1.0000	**0.8546**	0.1473	Slight overfitting.
3	**Decision Tree (Tuned)**	0.9887	**0.8269**	0.9887	**0.8263**	0.1618	Regularization gain (+2.30% F_1).
4	**Decision Tree (Default)**	1.0000	**0.8039**	1.0000	**0.8033**	0.1961	Severe overfitting (unlimited depth).
5	**KNN (Tuned – LDA)**	1.0000	**0.5698**	1.0000	**0.5726**	0.4302	Massive gain (+8.58% F_1 via LDA).
6	**KNN (Default – PCA)**	0.7495	**0.4840**	0.7573	**0.4977**	0.2655	Severe underfitting / Metric distortion.

2.5.1 Conclusions and Key Project Takeaways. In this section, we developed a complete supervised machine learning workflow to classify encrypted HTTPS network flows into 26 distinct domain names. The main conclusions and insights from our experiments are:

(1) **Effectiveness of Ensemble Methods: Random Forest (Tuned)** emerged as the top-performing model across all metrics, achieving a Test Weighted F_1 of **0.8537** and Test Accuracy of **0.8556**. By combining 300 decorrelated decision trees, Random Forest captures the complex, non-linear relationships in network measurements while effectively mitigating the severe overfitting seen in individual decision trees.
(2) **Importance of Regularization and Tree Depth Limits:** Unconstrained Decision Trees suffered from high variance, memorizing single-sample noise in the training set ($\Delta F_1 = 0.1961$). Restricting the tree depth to max_depth = 20 and tuning the split criterion provided a solid +2.30% test improvement, confirming the importance of pruning when working with noisy traffic measurements.

(3) **Supervised vs. Unsupervised Dimensionality Reduction:** Distance-based classifiers like KNN struggle in high-dimensional tabular spaces. Default KNN with unsupervised PCA performed poorly ($F_1 = 0.4840$) because PCA projects along directions of maximum data volume rather than domain separation. Replacing PCA with supervised Linear Discriminant Analysis (LDA) and using Cosine distance weighting yielded a substantial improvement (+8.58%), raising the test score to 0.5698.
(4) **Limits of Passive Transport-Layer Classification:** The in-depth error analysis proved that while distinct services (such as Spotify media streaming or real-time tracking beacons) produce unmistakable transport signatures ($AUC > 0.99$), services with identical underlying web architectures (like Taboola and Outbrain recommendation widgets) produce nearly identical packet and byte distributions. When physical traffic dynamics overlap and encryption conceals application headers, transport-level features alone reach a natural discriminative boundary.

These findings confirm that supervised machine learning can successfully identify encrypted web traffic patterns with high accuracy, provided that proper feature engineering, leakage prevention, and model regularization are systematically applied.

3 Section 3 – Unsupervised Learning – Domain Name Clustering

The objective of this section is to group domain names that generate similar HTTPS traffic patterns, independently of semantic service labels. Since the original dataset is flow-based, the first step is to construct one representative vector for each domain. The executed notebook contains **26 domain-level samples**.

3.1 Domain-level representation

3.1.1 Feature aggregation. The aggregation is designed to retain three complementary aspects of each domain: its *typical behaviour*, its *variability across flows*, and the presence of *extreme or bursty observations*. Client- and server-side quantities are kept separate because traffic direction remains informative even when the payload is encrypted.

Table 11. Domain-level feature aggregation used for clustering.

Feature group	Aggregation	Motivation
Client/server bytes	mean, std, max	Average transferred volume, variability and unusually large transfers.
Client/server packets	mean, std	Typical packet volume and dispersion.
Client/server RTT	mean, std, min, max	Typical latency, variability and extreme delay values.
Flow duration	mean, max, std	Typical session length, long-lived flows and variability.
Retransmissions, resets, zero-window, first message size, out-of-order packets, TLS status	mainly mean	Average intensity/frequency of connection-level behaviours.
Server MSS, window scale	max	Upper TCP settings observed for the domain.
Client-IP observations	count	Number of observed flow records associated with the domain; an activity/popularity measure, not a count of unique users.

A single mean would hide relevant information: two domains can have similar average byte volume but very different variance, RTT range or occasional large transfers. The additional statistics retain these differences while keeping the representation compact enough for only 26 domain-level observations.

3.1.2 Pre-processing and PCA. After aggregation, infinite values are replaced with missing values and then filled with zero. The features have very different units and ranges, therefore the domain-level matrix is standardized with `StandardScaler`. PCA is then applied, retaining **10 principal components**. In the executed notebook, the first five components explain **75.7%** of the variance. Clustering is performed in the full 10-dimensional PCA representation; 2D and 3D projections are used only for visualization.

This step is useful because the aggregated statistics are partially correlated. PCA provides a more compact Euclidean space while preserving the dominant traffic variation.

3.2 K-Means clustering

K-Means is the main partitioning algorithm. Each domain is assigned to the closest centroid by minimizing the within-cluster sum of squared Euclidean distances,

$$\min_{C_1,\dots,C_k} \sum_{j=1}^{k} \sum_{x_i \in C_j} \|x_i - \mu_j\|_2^2. \tag{2}$$

3.2.1 Initialization and choice of k. The number of clusters is explored for $k = 3, \ldots, 12$ using two internal criteria: **inertia/Elbow**, which measures within-cluster squared distance, and the **Silhouette score**, which measures cohesion and separation. For model selection K-Means uses the default `k-means++` initialization and `n_init=50`. Six random states (15, 42, 73, 11, 99 and 3) are evaluated to check stability rather than to select the most favourable initialization.

The Elbow result is stable: it selects $k = 7$ in five runs and $k = 8$ in one. The Silhouette optimum is less stable: $k = 8$ is preferred in four runs, while one run favours $k = 9$ and one favours $k = 3$. For the representative run with `random_state=15`, the elbow is at $k = 7$ and the maximum Silhouette occurs at $k = 9$. At $k = 7$, the model-selection run gives inertia 196.86 and Silhouette 0.285.

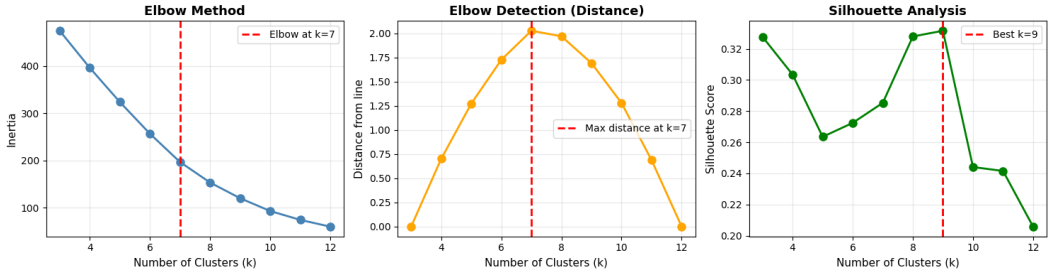
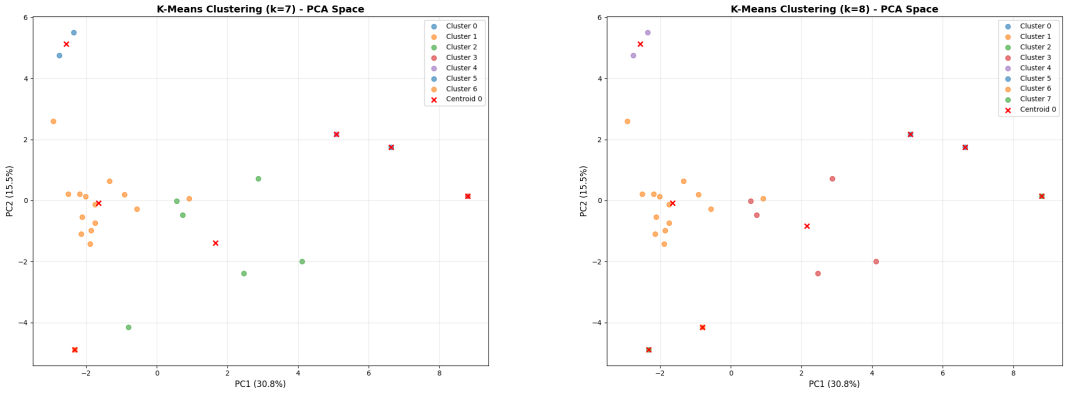


Fig. 11. K-Means model selection for `random_state=15`: inertia, geometric elbow detection and Silhouette score.

The important result is not a single "perfect" k, but the shape of the data. Silhouette values remain moderate because many domains occupy the same dense region of the PCA space. Their aggregate HTTPS/TCP behaviour is similar, so increasing k often isolates a few peripheral domains rather than revealing many equally separated families.

3.2.2 Observed partitions. The notebook explicitly visualizes $k = 7, 8$. The visualization helper uses `k-means++`, `random_state=15` and `n_init=10`. With $k = 7$, inertia is 201.91 and Silhouette is 0.316; the cluster sizes are 14, 6, 2, 1, 1, 1, 1. With $k = 8$, inertia decreases to 153.38 and Silhouette rises to 0.328, with cluster sizes 14, 5, 2 and five singletons.



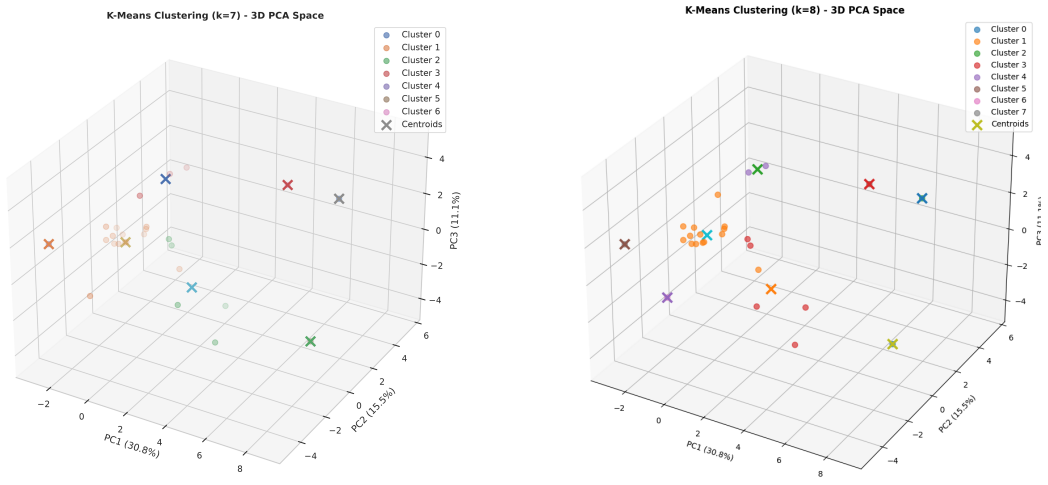
(a) $k = 7$. (b) $k = 8$.

Fig. 12. K-Means partitions projected onto PC1–PC2. The fitted models use all 10 retained PCs.

3.2.3 Three-dimensional PCA view. To make the geometry easier to inspect, the notebook also shows K-Means in the first three principal components. The 3D helper refits the model with `random_state=15` and `n_init=50`; it compares a coarse $k = 7$ partition with the finer $k = 8$ solution. PC1, PC2 and PC3 explain approximately 30.8%, 15.5% and 11.8% of the variance, respectively, so the 3D view represents about 58% of the total variation.

The 3D projection makes the same structure visible more clearly: most domains remain concentrated in a common region, while a limited number of samples are far from the central mass. The $k = 8$ solution therefore gains detail mainly by creating singleton clusters around these peripheral points. This is consistent with the moderate Silhouette values and explains why the number of clusters is not unambiguously determined by a single metric.

For $k = 8$, the five-domain group contains `giphy.com`, `redd.it`, `reddit.com`, `slack-edge.com` and `taboola.com`; the pair `twitch.tv/twitchcdn.net` is also separated. The large cluster contains 14 domains, while `scdn.co`, `vimeocdn.com`, `_other`, `ftcdn.net` and `fastly-insights.com` appear as singletons.



(a) $k = 7$: coarse partition.

(b) $k = 8$: finer partition.

Fig. 13. Three-dimensional K-Means visualization in the PC1–PC2–PC3 space. Centroids are marked with crosses.

3.3 DBSCAN

DBSCAN is used as a complementary density-based method, mainly to identify atypical domains. Unlike K-Means it does not require a predefined number of clusters. The notebook uses $\varepsilon = 6$ and `min_samples=3`; the radius is chosen heuristically from the scale of the K-Means point-to-centroid errors, while the small `min_samples` is appropriate for only 26 domain-level samples.

Table 12. DBSCAN result in the 10-dimensional PCA space.

Result	Count	Interpretation
Dense clusters	1	One dominant density region.
Core points	20	Most domains lie inside the same dense component.
Border points	1	`reddit.com`.
Noise points	5	`_other`, `fastly-insights.com`, `ftcdn.net`, `scdn.co`, `vimeocdn.com`.

This is the clearest evidence that the domain representations are highly similar: **20 of 26 domains are core points of a single DBSCAN component**. Hence most points are connected through dense neighbourhoods instead of being separated by low-density regions. The five noise domains are exactly the five singleton profiles isolated by the $k = 8$ K-Means solution, providing useful agreement between two different clustering principles.

3.4 Hierarchical clustering

Ward hierarchical clustering is applied to the same PCA representation as a structural cross-check. Ward linkage recursively merges groups so as to minimize the increase in within-cluster variance. The notebook inspects cuts at $k = 2, 7, 8$; the dendrogram is not treated as ground truth, but as an additional view of the geometry.

The dendrogram supports the same overall picture: a large branch contains many similar domains, while a few branches remain separated until larger linkage distances. It also reveals local relationships such as the close `twitch.tv/twitchcdn.net` pair.

3.5 Coarse interpretation of the detected groups

The last step is a *post-hoc* semantic interpretation. These names are not provided to the clustering algorithms; they are introduced only to summarize recurring traffic patterns observed after the analysis.

The interpretation is intentionally coarse. The Social group corresponds directly to the five-domain K-Means group, while the Noisy group coincides with the five $k = 8$ singletons and the five DBSCAN noise points. The liveStream label combines the clear Twitch pair with `chartbeat.com`; the remaining large group is summarized as Tracker. Thus the semantic names make the structure easier to discuss without changing the unsupervised nature of the analysis.

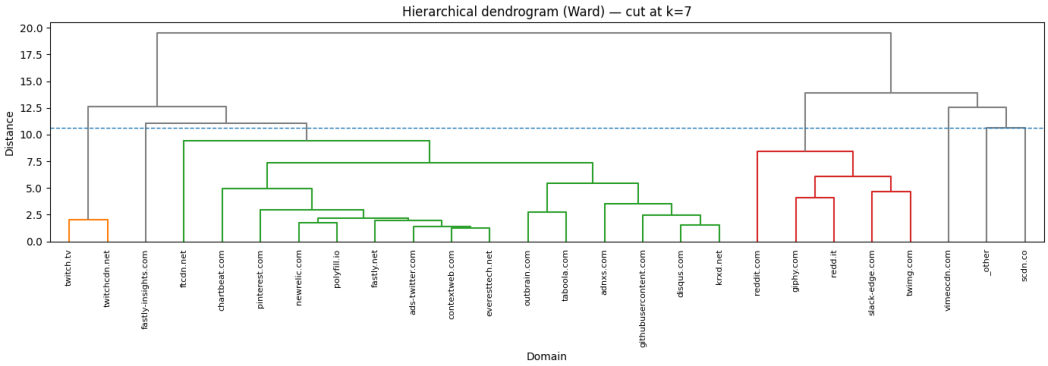
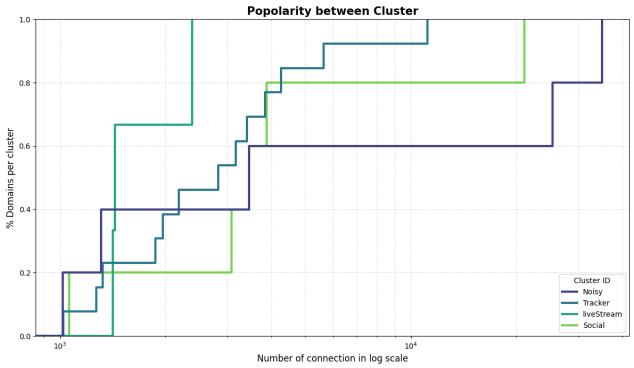


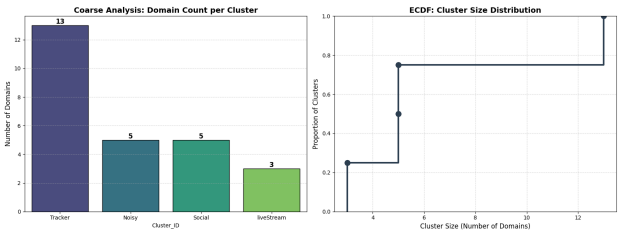
Fig. 14. Ward hierarchical clustering cut at approximately $k = 7$.

Table 13. Coarse semantic interpretation used in the notebook.

Family	Size	Domains / interpretation
Tracker	13	`adnxs.com`, `ads-twitter.com`, `contextweb.com`, `disqus.com`, `everesttech.net`, `fastly.net`, `githubusercontent.com`, `krxd.net`, `newrelic.com`, `outbrain.com`, `pinterest.com`, `polyfill.io`, `twimg.com`.
Social	5	`giphy.com`, `redd.it`, `reddit.com`, `slack-edge.com`, `taboola.com`.
liveStream	3	`chartbeat.com`, `twitch.tv`, `twitchcdn.net`.
Noisy	5	`vimeocdn.com`, `scdn.co`, `fastly-insights.com`, `ftcdn.net`, `_other`.



(a) ECDF of domain activity/popularity.



(b) Family sizes and size ECDF.

Fig. 15. Coarse analysis of the interpreted families. Popularity is measured through the number of observed client-IP flow records and is shown on a logarithmic scale.

The ECDF complements PCA distance with an interpretable traffic quantity. Domains that are geometrically close can still differ substantially in the number of observed connections, so popularity is useful to characterize the groups but is not a replacement for the multidimensional clustering representation.

3.6 Conclusions

The unsupervised analysis does not reveal many sharply separated domain families. Instead, all three methods indicate a **large common traffic region together with a small number of more distinctive profiles**. This behaviour is expected because the aggregated HTTPS/TCP statistics of many services are similar: once each domain is summarized through averages, variability and extrema, a substantial part of the flow-level differences is compressed into overlapping domain-level vectors.

K-Means therefore produces a stable elbow around $k = 7$ but only moderate Silhouette values, and finer partitions mostly isolate peripheral points. The 2D and 3D PCA views make this geometry visible. DBSCAN reinforces the conclusion even more clearly: 20/26 domains are core points in one dense component, one domain is a border point and only five are noise. Ward clustering gives the same qualitative picture from a hierarchical perspective.

The final result is therefore better described as **one dominant shared behaviour, a few coherent sub-groups and a small atypical tail**, rather than as a set of perfectly separated classes. The coarse Tracker/Social/liveStream/Noisy interpretation provides a compact semantic description of these patterns while remaining a post-hoc analysis of the unsupervised clustering results.

4 Section 4 – Regression: Estimating Transmitted Bytes and RTT

We considered the two flow-level regression problems: estimating server-to-client payload volume, `_s_bytes_all`, and average server-to-client round-trip time, `_s_rtt_avg`. We applied Lasso, Decision Tree, and Random Forest to both targets. This common set provides a regularized linear model, a flexible nonlinear model, and a tree ensemble, allowing underfitting, overfitting, and the effect of tuning to be compared under the same protocol.

4.1 Experimental Protocol

The supplied training and test files were kept separate. Preprocessing was derived from the official training set and applied to the test set. Hyperparameter and model-family selection used three-fold cross-validation (CV) within the preprocessed training data; fixed configurations were then refitted on all task-specific training flows. Official-test results therefore estimate unseen-flow performance but do not determine the declared model.

Common preprocessing removed duplicate observations, missing or constant columns, client identifiers, timestamps, and the domain label. Non-negative predictors received a `log1p` transformation, one variable from each pair with absolute training correlation above 0.985 was removed, and `RobustScaler` was fitted on training data. Both targets span several orders of magnitude, so the regressors learned `log1p` of the target through `TransformedTargetRegressor`. Predictions were transformed back with `expm1`, clipped at zero, and evaluated in the original physical unit.

Mean Absolute Error (MAE) is the primary metric because it expresses average absolute error directly in bytes or milliseconds. Root Mean Squared Error (RMSE) gives additional weight to rare large errors, while R^2 measures improvement over a constant-mean predictor under squared loss. High training performance followed by a large test deterioration indicates overfitting; weak performance on both splits indicates underfitting.

Default models used every training flow. Tuning used shuffled three-fold CV on all 147,863 byte observations and 147,832 nonzero-RTT observations; each configuration was evaluated once per validation fold and ranked by mean MAE. No row-sampling or final-fitting limit was used.

For Lasso we searched $\alpha \in \{0.0001, 0.001, 0.01, 0.1, 1, 10\}$. For the Decision Tree we searched `max_depth`$\in \{8, 16, \text{None}\}$ and `min_samples_leaf`$\in \{5, 25, 75\}$. Random Forest used 100 trees in both default and tuned experiments; tuning searched depth in $\{12, \text{None}\}$, leaf size in $\{5, 25\}$, and `max_features`$\in$ {sqrt, 0.5}. Fixing 100 trees avoids confounding the comparison with a different ensemble size. The forest grid deliberately tests more regularized configurations and does not include the exact default combination, so tuning is not guaranteed to improve the default forest.

Within each family, CV selects hyperparameters; the smallest CV MAE across the three tuned families identifies the declared model. The best test result is reported only retrospectively to characterize behavior on the official evaluation distribution.

4.2 Task 1: Predicting Server-to-Client Bytes

4.2.1 Data Preparation. The target `_s_bytes_all` is server-to-client payload volume including retransmissions and is measured in bytes. The target and `_s_bytes_uniq` were removed from the predictors as required. We additionally excluded `_s_appdataB`, a near-direct application-payload counter that would make the intended prediction problem less meaningful. After preprocessing, 88 predictors remained for 147,863 training and 114,580 test flows.

The target is strongly right-skewed: training median and mean are 7,617 and 359,311 bytes, versus 7,170 and 346,466 on test. More importantly, the training maximum is 4.55×10^8 bytes, whereas test reaches 2.62×10^9. This upper-tail shift plausibly disadvantages tree models, whose piecewise-constant predictions cannot extrapolate naturally beyond values represented in training leaves.

Table 14. Default models for `_s_bytes_all`. Errors are in bytes.

Model	Train MAE	Test MAE	Test RMSE	Train R^2	Test R^2
Lasso	343,965	334,365	9,281,318	0.0067	0.0039
Tree	< 1	44,011	7,718,738	1.0000	0.3110
Forest	2,642	**37,463**	**7,286,076**	0.9927	**0.3861**

4.2.2 Default Models. Table 14 shows opposite failure modes. Default Lasso has R^2 close to zero on both splits: the small gap represents stable underfitting, not successful generalization. The unrestricted Decision Tree reproduces training observations almost exactly, but test R^2 falls to 0.3110. Random Forest has the best default test metrics, yet R^2 decreases from 0.9927 to 0.3861. Both tree methods therefore overfit, although forest averaging reduces test error.

Table 15. Tuned byte models refitted on all training flows. The asterisk marks training-only CV selection; bold test values mark the best retrospective test result.

Model	CV MAE	Train MAE	Test MAE	Test RMSE	Train R^2	Test R^2
Lasso	19,680	19,658	**23,889**	**1,090,812**	0.9926	**0.9862**
Tree	12,080	6,084	43,433	7,403,349	0.9738	0.3662
Forest*	**10,970**	7,162	46,481	7,747,562	0.9606	0.3059

4.2.3 Cross-Validation and Tuned Models. CV selected $\alpha = 0.0001$ for Lasso, depth 16 with leaf size five for the tree, and depth 12, leaf size five, and `max_features`= 0.5 for the forest. Table 15 shows that Random Forest has the lowest CV MAE (10,970 bytes), followed by Decision Tree (12,080) and Lasso (19,680). The training-only rule therefore selects Random Forest.

The official test set produces a pronounced ranking reversal. The selected forest reaches 46,481-byte MAE and $R^2 = 0.3059$, whereas tuned Lasso reaches 23,889-byte MAE and $R^2 = 0.9862$. Lasso also has an R^2 gap of only 0.0064, while tree and forest gaps remain 0.6076 and 0.6547. Thus Lasso provides by far the best observed test generalization, but adopting it only after inspecting official-test metrics would change the predefined selection rule.

This disagreement is interpretable rather than contradictory. Trees achieve lower validation error within the training distribution but are bounded by values represented in their terminal leaves. Lasso can extend a learned relationship beyond that range. In particular, volume is approximately related to packet or segment count multiplied by typical size; after log transformation, such multiplicative structure becomes approximately additive and therefore well suited to a linear model. Its high R^2 may thus reflect an almost accounting-level relationship among retained flow counters rather than recovery of complex application behavior. The evidence is consistent with upper-tail shift and different extrapolation behavior, but without tail-stratified errors it does not establish that the shift alone causes the ranking reversal.

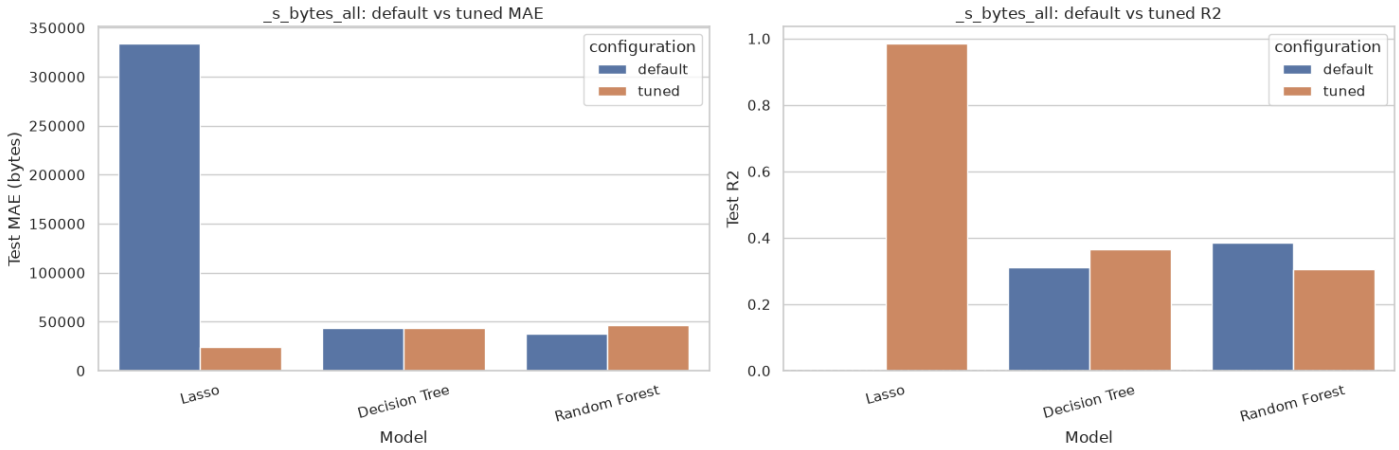


Figure 16: Default and tuned official-test performance for byte prediction. Weakening Lasso regularization produces the strongest test improvement, while both tuned tree-based models retain large train–test gaps.

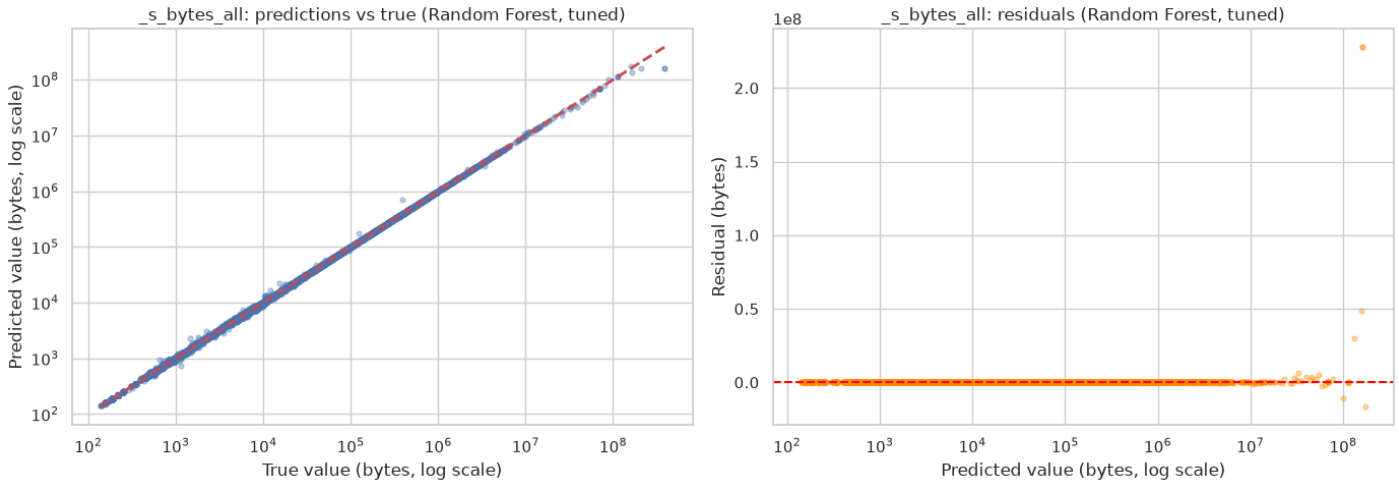


Figure 17: Official-test predictions and residuals for the CV-selected byte Random Forest. Rare upper-tail errors dominate RMSE and reduce R^2.

4.3 Task 2: Predicting Average RTT

4.3.1 Data Preparation. The target `_s_rtt_avg` is the average server-to-client RTT between a data segment and its acknowledgement and is measured in milliseconds. Values equal to zero were removed separately from training and test because they indicate absence of a usable RTT estimate. The target and every other RTT-related field-client and server-side counts, minima, maxima, means, and standard deviations were excluded to prevent reconstruction from alternative summaries of the same latency measurements. This left 80 predictors, 147,832 training flows, and 114,555 test flows.

RTT is extremely right-skewed. The training median is 9.91 ms and mean is 426.45 ms; the test median is 9.12 ms and mean is 511.03 ms. The 99th percentiles exceed 14,800 ms in both splits, and the test maximum is 116,853.67 ms. After removing direct RTT summaries, retained statistics describe these effects only indirectly and do not provide direct measurements of path length, routing, queueing, or transient congestion. Large RTT errors are therefore plausible even when predictions are reasonably accurate for the central portion of the distribution.

Table 16. Default models for `_s_rtt_avg`. Errors are in milliseconds.

Model	Train MAE	Test MAE	Test RMSE	Train R^2	Test R^2
Lasso	422.34	507.76	3,212.50	−0.0183	−0.0246
Tree	< 0.01	507.45	2,989.10	1.0000	0.1129
Forest	184.23	**377.19**	**2,638.57**	0.7115	**0.3088**

4.3.2 Default Models. Default Lasso underfits and is worse under squared loss than predicting the split mean. The unrestricted tree memorizes training data and loses most of its fit on test data. Random Forest is the strongest default RTT model, although its R^2 gap of 0.4027 still indicates overfitting. For every model, RMSE is several times larger than MAE because a small number of high-latency flows generate extreme residuals.

Table 17. Tuned RTT models refitted on all nonzero-target training flows. The asterisk marks training-only CV selection.

Model	CV MAE	Train MAE	Test MAE	Test RMSE	Train R^2	Test R^2
Lasso	415.90	415.99	501.16	3,241.20	−0.0118	−0.0430
Tree*	**288.96**	194.18	**390.93**	**2,569.94**	0.6931	**0.3443**
Forest	310.32	259.76	399.88	2,764.15	0.5085	0.2414

4.3.3 Cross-Validation and Tuned Models. CV selected Lasso with $\alpha = 0.01$, a Decision Tree of depth 16 with leaf size five, and an unrestricted Random Forest with leaf size five and `max_features=` 0.5. The Decision Tree obtains the lowest CV MAE, 288.96 ms, and is therefore selected. The official test ranking among tuned models agrees: the tree has 390.93-ms MAE, 2,569.94-ms RMSE, and $R^2 = 0.3443$.

Depth and leaf constraints substantially improve the Decision Tree: relative to default, test MAE falls by 22.96%, test R^2 rises by 0.2313, and the R^2 gap falls from 0.8871 to 0.3488. The model still overfits, but no longer creates one-observation leaves responsible for the default tree's perfect training score. Tuned Lasso remains underfit, while tuned forest is inferior to the selected tree under CV MAE, test MAE, RMSE, and test R^2.

Default Random Forest has a lower official-test MAE than the selected tuned tree (377.19 versus 390.93 ms). It was evaluated as the untuned baseline but was not included as a candidate in the cross-model CV ranking. It therefore cannot be promoted using its test result without redefining the selection procedure. Its lower test MAE is nevertheless relevant when assessing whether tuning was beneficial: tuning improves the tree decisively but not every method, and tuned forest test MAE is 6.02% higher than its default counterpart.

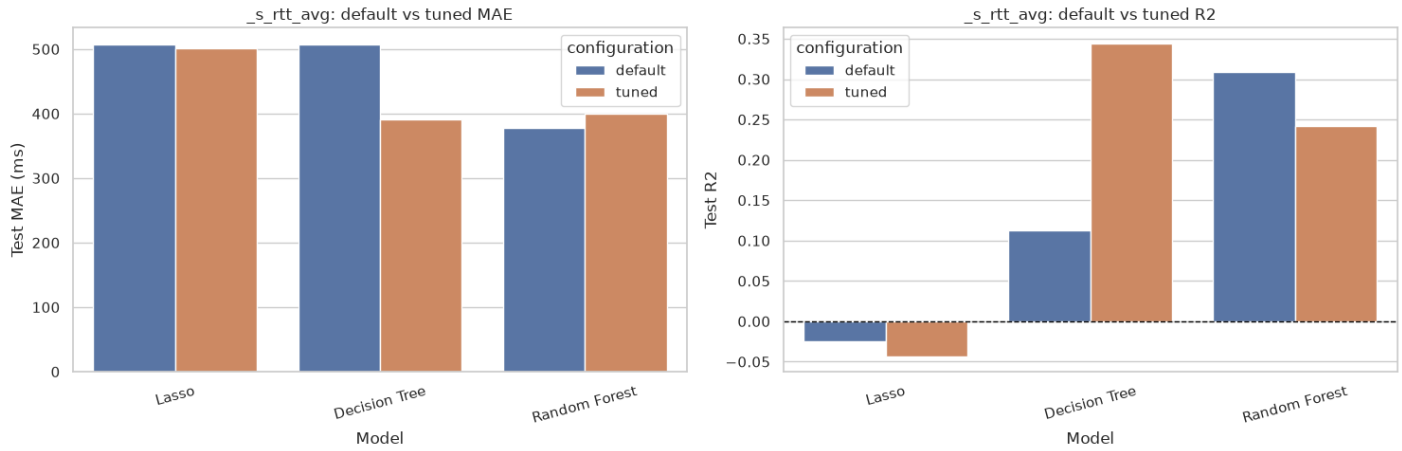


Figure 18: Default and tuned official-test RTT performance. Constraining the Decision Tree produces the clearest improvement, whereas Lasso remains underfit and tuned forest does not improve on default.

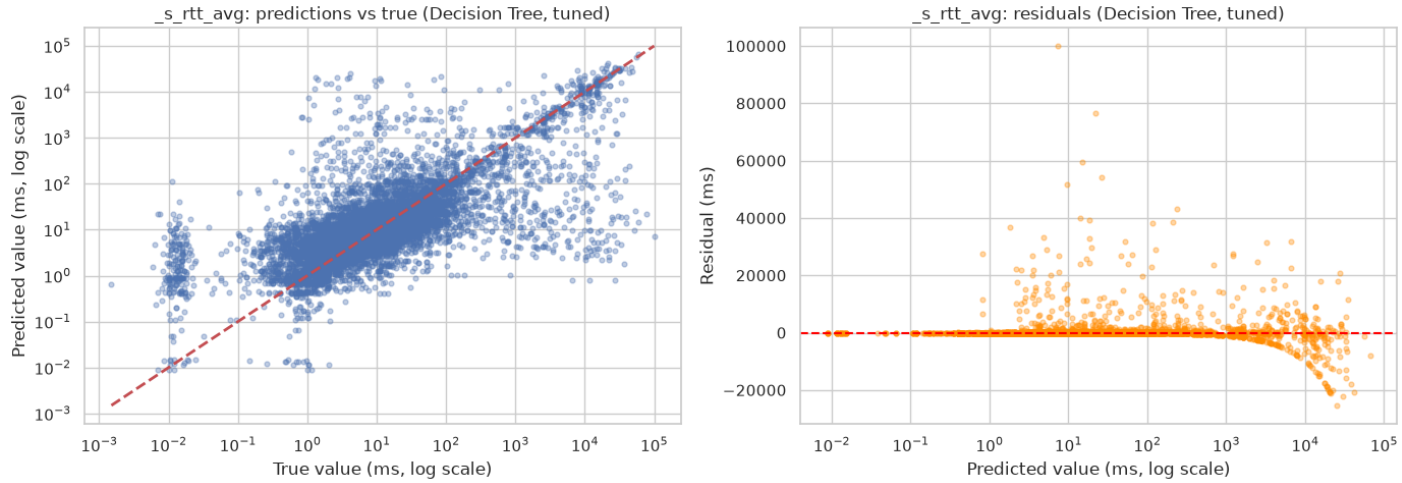


Figure 19: Official-test predictions and residuals for the CV-selected depth-16 RTT Decision Tree. Broad dispersion is consistent with limited latency information and the extreme upper tail.

4.4 Comparison of the Two Regression Problems

The absolute MAEs use different physical units. For CV-selected models, byte prediction has MAE 46,481 bytes, while RTT prediction has MAE 390.93 ms. Thus `_s_bytes_all` has the numerically higher MAE, measured in bytes, but the magnitudes cannot be interpreted as if they shared a scale.

The scale-aware comparison is metric-dependent. Relative to each test-target mean, selected-model MAE is 13.42% for bytes and 76.50% for RTT, while test R^2 is slightly higher for RTT (0.3443 versus 0.3059) because the byte forest is strongly affected by the upper tail. Across all predefined tuned models, byte Lasso reaches 6.90% normalized MAE and $R^2 = 0.9862$. Since heavy tails affect both means, these ratios are interpreted with R^2, not alone. Overall, byte volume is more predictable from the retained flow counters when extrapolation is permitted; absolute MAE and selected-model metrics alone do not give a uniform difficulty ranking.

4.5 Limitations and Conclusions

Complete-data three-fold CV removes the earlier sampling limitation, but random folds cannot represent targets beyond training support. This matters for bytes, where the test maximum is substantially larger and the CV ranking does not transfer. Moreover, correlation pruning and scaling were fitted once on the complete training split before CV rather than refitted inside each fold. No test information was used, but validation folds contributed to these unsupervised estimates, so CV errors may be mildly optimistic. Finally, the finite grids cover few configurations and the forest grid excludes the exact default combination; broader, temporal, or tail-aware validation remains future work.

The log-target transformation reduces the fitting influence of extremes, whereas metrics are computed after inversion and remain upper-tail sensitive. It is intended to stabilize learning across orders of magnitude; the present experiment does not isolate its causal effect.

In conclusion, default Lasso underfits both targets, unrestricted trees overfit, and default Random Forest is the strongest baseline. CV selects Random Forest for bytes and a depth-16 tree for RTT. The latter leads among tuned test candidates, although default Random Forest has slightly lower test MAE. For bytes, tuned Lasso instead gives the best observed test result, consistent with log-linear extrapolation under the shifted upper tail. Despite incomparable absolute units, the full evidence indicates that byte volume is more predictable from the retained features than RTT, although the CV-selected byte forest does not exploit that predictability as effectively.